

▼ 无监督学习

▼ 聚类 (Clustering Algorithms)

▼ K-means

- 步骤 (迭代过程)
- 代价函数
- K-Means 初始化

▼ K 值 (Number of Clusters)

- 肘部法则 (Elbow Method)

▼ 异常

- 高斯分布的异常检测 (Gaussian Distribution Model)
- 评估异常检测

▼ 推荐程序

▪ 特征推荐法

▼ 协同过滤法 (Collaborative Filtering)

- 代码片段

▪ 二元标签

▪ 均值归一化 (Mean Normalization)

▼ 内容过滤法

- 偏好提取
- 代码片段

▼ 强化学习

▼ 核心过程

- 折扣因子 (Discount Factor)
- 策略 (Policy, π)
- 回报
- Bellman 方程

- 阿波罗计划

无监督学习

聚类 (Clustering Algorithms)

聚类是一种无监督学习，其是在没有预先标记的情况下，将训练集中的样本分成若干个组（称作簇 / **Clusters**）。同一簇内的样本彼此相似，而不同簇的样本相似性低。

K-means

将 m 个样本点分成 K 个簇，每个样本点都归类于离它最近的簇的中心（即质心 / **Centroid**）。

步骤 (迭代过程)

K-Means 通过迭代的方式逐步优化簇的分配和质心的位置。

1. 随机初始化质心 (Random Initialization):

- 随机选择 K 个样本点作为初始的簇中心 ($\mu_1, \mu_2, \dots, \mu_K$)。

2. 簇分配阶段 (Cluster Assignment Step / E-step):

- 遍历所有训练样本 $x^{(i)}$ 。
- 对于每个样本 $x^{(i)}$ ，统计它到所有 K 个质心 ($\mu_1, \dots, \mu_K$) 的距离。
- 将 $x^{(i)}$ 归到离它最近的质心所在的簇。
- 记录下 $x^{(i)}$ 所归的簇的下标 $c^{(i)}$ 。

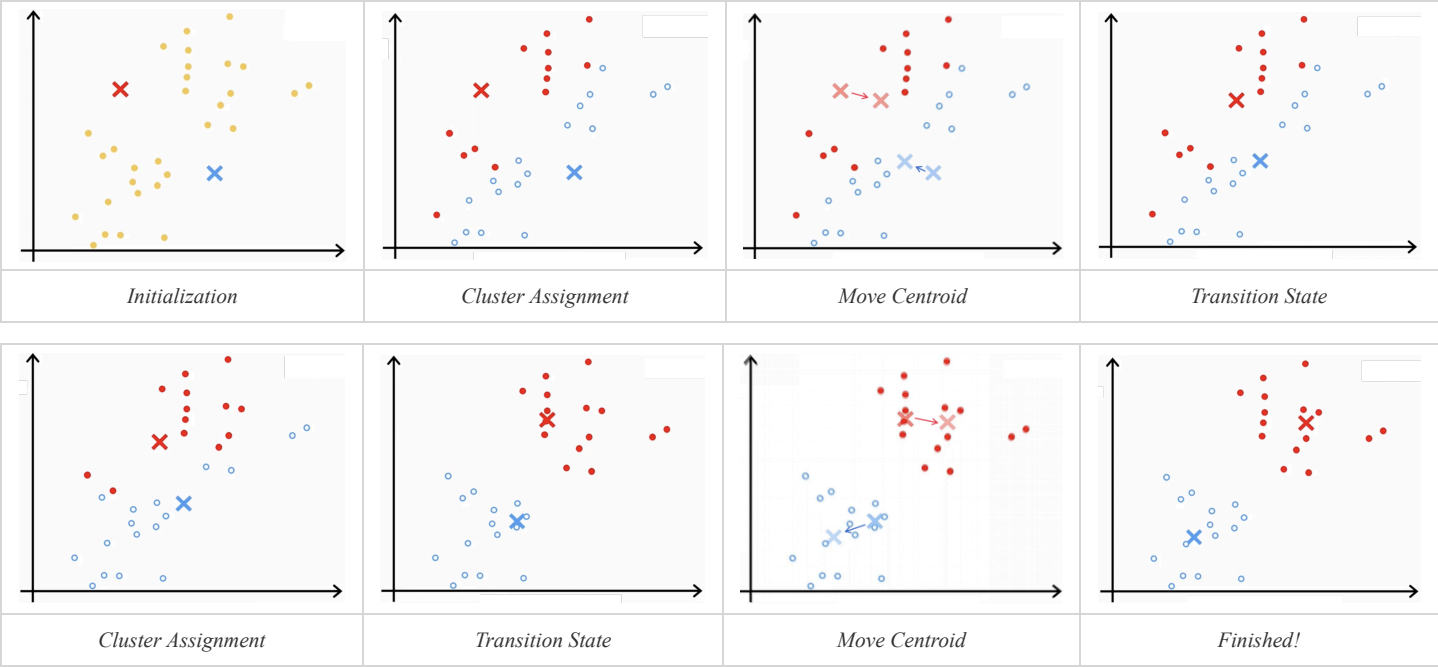
3. 移动质心阶段 (Move Centroid Step / M-step - 最大化步骤):

- 遍历所有 K 个簇。
- 对于每个簇 j ，得到其质心 μ_j 。新的质心是该簇中所有被分配到它的样本的平均值。

$$\mu_j = \frac{1}{\text{size}(C_j)} \sum_{i \in C_j} x^{(i)}$$

其中 C_j 是分配到簇 j 的所有样本的集合。

4. 迭代：
- 执行步骤2、3，直到质心不再显著移动（即收敛），或者达到预设的最大迭代步。



代价函数

K-Means 的代价是点到其所簇心之间的平方距离之和。

$$J(c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K) = \frac{1}{m} \sum_{i=1}^m \|x^{(i)} - \mu_{c^{(i)}}\|^2$$

- m : 训练样本。
- $x^{(i)}$: i_{th} 个训练样本。
- $c^{(i)}$: 样本 $x^{(i)}$ 归到的簇。
- $\mu_{c^{(i)}}$: 样本 $x^{(i)}$ 所归到的簇心。

K-Means 初始化

K-Means 对**初始簇心的选择**非常依赖。不同的初始化中心可能使其收敛到不同的**局部最优解**，而不是全局最优。

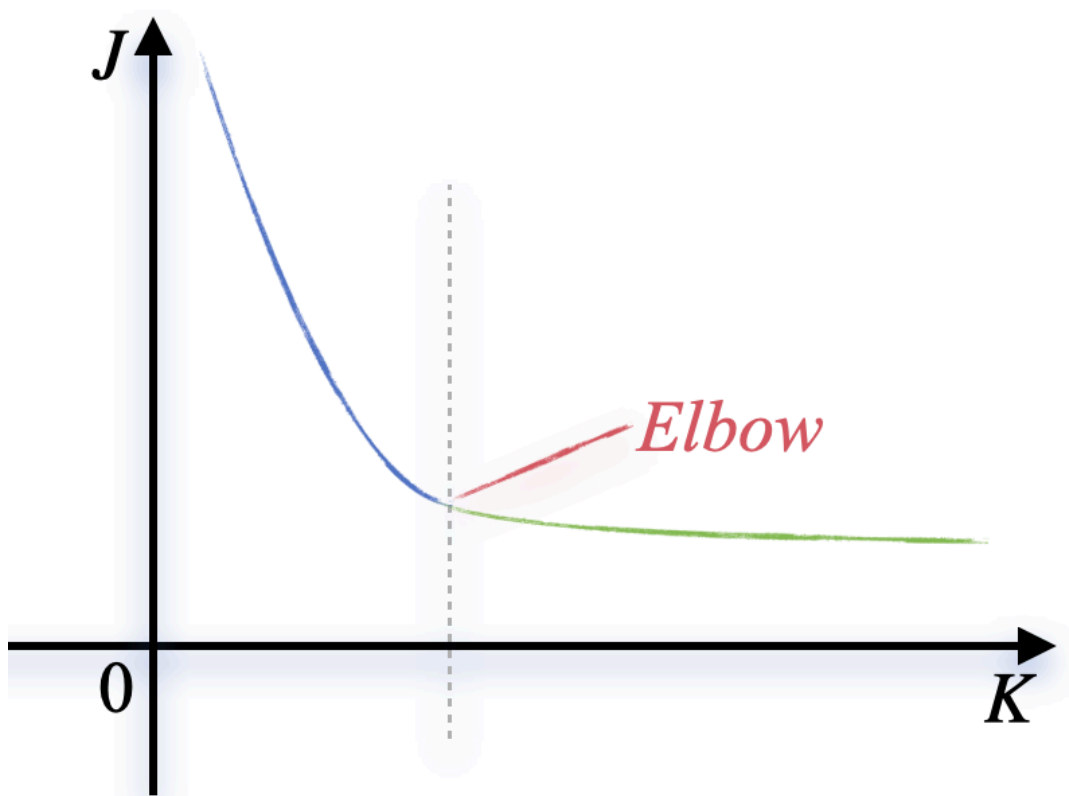
通常来说，K-Means 初始中心随机在所有样本中选 K 个样本点，而不是在空间中任选 K 个点，*and run it more*，选 J 值最小的聚类结果作为最终的聚类。

K 值 (Number of Clusters)

如何选最佳的 K 值，以得到最好的聚类效果呢？

肘部法则 (Elbow Method)

以下是 $J - K$ 的曲线，部分聚类中，该曲线上存在“**肘部**”（或“拐点”）。在这个点之后， K 增加， J 的下降收益小，为此拐点通常是 K 值选择。



但，有时曲线平滑下降，无“肘部”，此时“现实才是检验 K 值的标准”，自己体验一下不同 K 值带来的效果吧！

异常

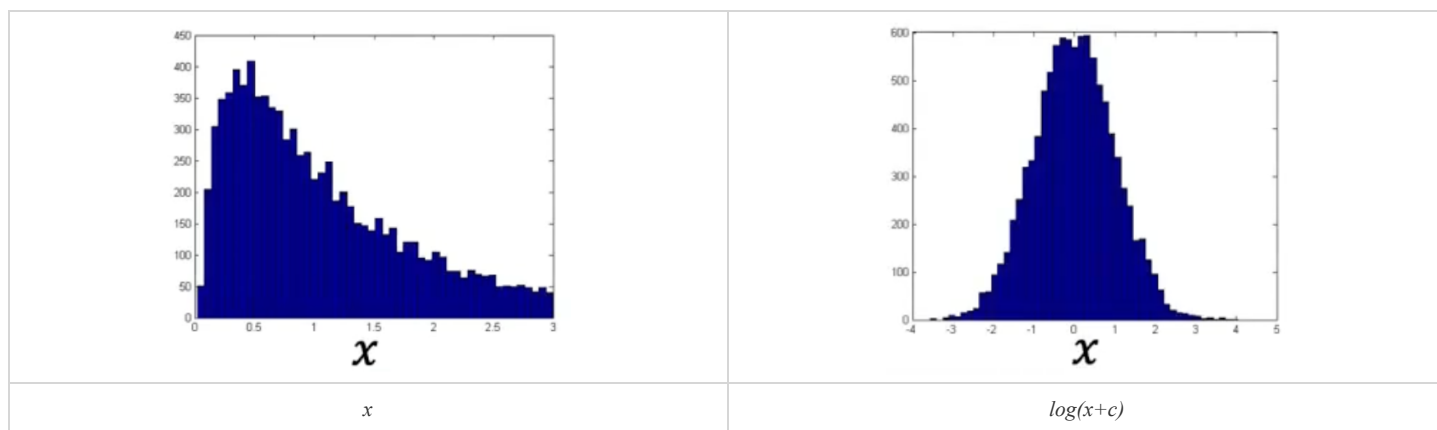
异常检测是识别训练集中与绝大部分样本点显著不同的观测值的过程。这些不同寻常的样本点称作**异常 (Anomalies)** 或 **离群点 (Outliers)**。

高斯分布的异常检测 (Gaussian Distribution Model)

对于符合正太分布的样本集，首先采取估计法（矩估计、极大估计等），得到 μ 和 σ^2 ，则异常点即高斯分布下具有**很低概率**的点。（有点类似于假设检验）

1. 特征选择：

- 选择能指示异常行为的特征 $x_1, x_2, \dots, x_n$ 。
- 注： 如果一些特征不服从高斯分布，可以进行**特征变换**，使其符合高斯分布（比如，取 $\log(x)$ 、平方根 $\text{sqrt}(x)$ 等）。



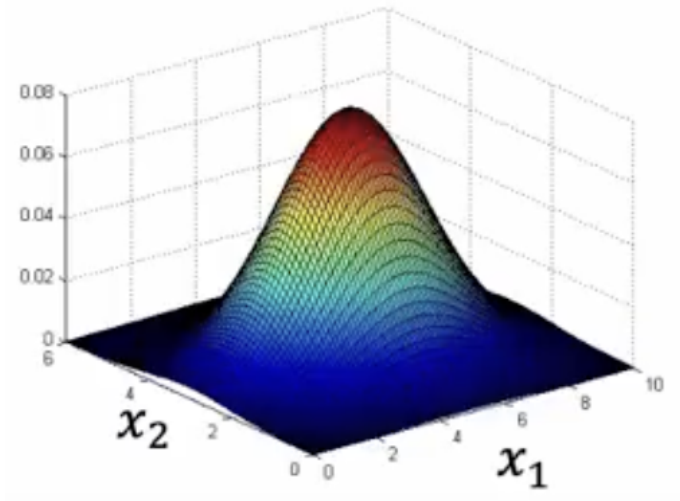
2. 参数估计 (Training Phase):

- 采取只包含正常样本的训练集来估计每个特征 x_j 的 μ_j 和 σ_j^2 。

$$\begin{cases} \mu_j = \frac{1}{m} \sum_{i=1}^m x_j^{(i)} \\ \sigma_j^2 = \frac{1}{m} \sum_{i=1}^m (x_j^{(i)} - \mu_j)^2 \end{cases}$$

3. 概率模型 $p(x)$:

- 假如特征是**相互独立的**（或近似独立）。
- 对于一个新的样本点 $x = [x_1, x_2, \dots, x_n]^T$ ，其概率 $p(x)$ 等于每个特征的 *probability density*^[1] 的乘积：



$$p(x) = \prod_{j=1}^n p(x_j; \mu_j, \sigma_j^2)$$

其中，单个特征 x_j 在高斯分布下的 *probability density function* 为：

$$p(x_j; \mu_j, \sigma_j^2) = \frac{1}{\sqrt{2\pi}\sigma_j} \exp\left(-\frac{(x_j - \mu_j)^2}{2\sigma_j^2}\right)$$

4. 阈值 ϵ ：
- 选择一个概率阈值 ϵ 。
 - 异常：如果 $p(x) < \epsilon$ ，则将 x 标记为异常；否则标记为正常。

评估异常检测

与之前一样，采取验证集来选择最佳的 ϵ 。

在异常检测中，绝大部分样本都是正常的，训练集中有众多正常样本，用于学习 μ, σ^2 ；验证集中包含部分正常样本和很少标记的异常样本，用于选择最佳阈值 ϵ ；测试集同样包含部分正常样本和很少标记的异常样本，来评估模型的泛化能力。

评估指标： $Precision = \frac{TP}{TP+FP}$ ， $Recall = \frac{TP}{TP+FN}$ ， $F_1\ Score = 2 \times \frac{Precision \times Recall}{Precision+Recall}$

推荐程序

推荐程序是一种信息过滤程序，旨在预测客户对物品的偏好，并向他们推荐可能有兴趣的物品。

特征推荐法

以电影推荐说明，假如样本中共有 n_u 个人， n_m 部电影， n 个新引入的特征， $r(i, j) = 1$ 即有人对电影作了评价，比如 $r(1, 1) = 1$ ， $r(3, 1) = 0$ ， $y^{(i, j)}$ 为电影评分，比如 $y^{(1, 1)} = 5$ ， $y^{(4, 1)} = 0$ ，且 j 共评价了 $m^{(j)}$ 部电影。

Movie	Alice(1)	Bob(2)	Carol(3)	Dave(4)	x_1 (romance)	x_2 (action)
Love at last	5	5	0	0	0.9	0
Romance forever	5	?	?	0	1.0	0.01
Cute puppies of love	?	4	0	?	0.99	0
Nonstop car chases	0	0	5	4	0.1	1.0
Swords vs. karate	0	0	5	?	0	0.9

现在来给 Alice 推荐一下 《Cute puppies of love》，推荐公式如下：

$$\hat{y}^{(i, j)} = \vec{w}^{(j)} \cdot \vec{x}^{(i)} + b^{(j)}$$

首先，由她已经看过的电影，假如已经得到 $w^{(1)} = [5 \ 0]$ ， $b^{(1)} = 0$ ，再由特征表可知 $x^{(3)} = \begin{bmatrix} 0.9 \\ 0 \end{bmatrix}$ ，此时， $\hat{y}^{(3, 1)} = \vec{w}^{(1)} \cdot \vec{x}^{(3)} + b^{(1)} = 4.95$ 。

由以上分析不难得到，以上推荐就等同于回归问题，目标就是由现有样本训练出每个人的 $\vec{w}^{(j)}$ 和 $b^{(j)}$ ，为此 j 的推荐电影如下：

$$\min_{w^{(j)}, b^{(j)}} J(w^{(j)}, b^{(j)}) = \frac{1}{2m^{(j)}} \sum_{i:r(i,j)=1} \left((w^{(j)}) \cdot x^{(i)} + b^{(j)} - y^{(i,j)} \right)^2 + \frac{\lambda}{2m^{(j)}} \sum_{k=1}^n (w_k^{(j)})^2$$

不过，在推荐程序中通常将 $m^{(j)}$ 删掉，也不影响 $\vec{w}^{(j)}$ 和 $b^{(j)}$ 的迭代，最后得到：

$$J(w^{(j)}, b^{(j)}) = \frac{1}{2} \sum_{i:r(i,j)=1} \left((w^{(j)}) \cdot x^{(i)} + b^{(j)} - y^{(i,j)} \right)^2 + \frac{\lambda}{2} \sum_{k=1}^n (w_k^{(j)})^2$$

将所有人的 $J(w^{(j)}, b^{(j)})$ 组合起来，一次性采取 *Gradient Descent* 得到所有的参量的最优解。

$$J(w^{(1)}, \dots, w^{(n_u)}, b^{(1)}, \dots, b^{(n_u)}) = \frac{1}{2} \sum_{j=1}^{n_u} \sum_{i:r(i,j)=1} \left((w^{(j)}) \cdot x^{(i)} + b^{(j)} - y^{(i,j)} \right)^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (w_k^{(j)})^2$$

协同过滤法 (Collaborative Filtering)

假如事先不知道电影的特征呢，又该如何进行推荐？

Movie	Alice(1)	Bob(2)	Carol(3)	Dave(4)	x ₁ (romance)	x ₂ (action)
Love at last	5	5	0	0	?	?
Romance forever	5	?	?	0	?	?
Cute puppies of love	?	4	0	?	?	?
Nonstop car chases	0	0	5	4	?	?
Swords vs. karate	0	0	5	?	?	?

再假如已经事先得到每个人的训练参量：

$$w^{(1)} = \begin{bmatrix} 5 \\ 0 \end{bmatrix}, \quad w^{(2)} = \begin{bmatrix} 5 \\ 0 \end{bmatrix}, \quad w^{(3)} = \begin{bmatrix} 0 \\ 5 \end{bmatrix}, \quad w^{(4)} = \begin{bmatrix} 0 \\ 5 \end{bmatrix}$$

$$b^{(1)} = 0, \quad b^{(2)} = 0, \quad b^{(3)} = 0, \quad b^{(4)} = 0$$

由于 $b^{(j)} = 0$ 为了简化，将 $b^{(j)}$ 先删掉，即：

$$\text{using } w^{(j)} \cdot x^{(i)} + \cancel{b^{(j)}}$$

由已有参量，反求出特征 $x^{(1)}$ 的值，其他特征一样：

$$\left. \begin{array}{l} w^{(1)} \cdot x^{(1)} \approx 5 \\ w^{(2)} \cdot x^{(1)} \approx 5 \\ w^{(3)} \cdot x^{(1)} \approx 0 \\ w^{(4)} \cdot x^{(1)} \approx 0 \end{array} \right\} \longrightarrow x^{(1)} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

所以，对于某个特征 $x^{(i)}$ 其 $J(x^{(i)})$ 如下：

$$J(x^{(i)}) = \frac{1}{2} \sum_{j:r(i,j)=1} \left((w^{(j)}) \cdot x^{(i)} + b^{(j)} - y^{(i,j)} \right)^2 + \frac{\lambda}{2} \sum_{k=1}^n (x_k^{(i)})^2$$

同样地，可以一次性学习所有特征：

$$J(x^{(1)}, x^{(2)}, \dots, x^{(n_m)}) = \frac{1}{2} \sum_{i=1}^{n_m} \sum_{j:r(i,j)=1} \left((w^{(j)}) \cdot x^{(i)} + b^{(j)} - y^{(i,j)} \right)^2 + \frac{\lambda}{2} \sum_{i=1}^{n_m} \sum_{k=1}^n (x_k^{(i)})^2$$

那么，到现在为止，什么是协同过滤？前面学了已知特征 $x^{(i)}$ 训 $w^{(j)}$ 和 $b^{(j)}$ ，和已知 $w^{(j)}$ 和 $b^{(j)}$ 训 $x^{(i)}$ 的方法，协同过滤就是将 $J(w^{(1)}, \dots, w^{(n_u)}, b^{(1)}, \dots, b^{(n_u)})$ 和 $J(x^{(1)}, x^{(2)}, \dots, x^{(n_m)})$ 组合起来，协同训练的方法。

$$\min_{\substack{w^{(1)}, \dots, w^{(n_u)} \\ b^{(1)}, \dots, b^{(n_u)} \\ x^{(1)}, \dots, x^{(n_m)}}} J(w, b, x) = \frac{1}{2} \sum_{(i,j):r(i,j)=1} \left((w^{(j)}) \cdot x^{(i)} + b^{(j)} - y^{(i,j)} \right)^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (w_k^{(j)})^2 + \frac{\lambda}{2} \sum_{i=1}^{n_m} \sum_{k=1}^n (x_k^{(i)})^2$$

同样可以对以上式子采取 *Gradient Descent* 来获得最优参量：

$$w_i^{(j)} = w_i^{(j)} - \alpha \frac{\partial}{\partial w_i^{(j)}} J(w, b, x)$$

$$b^{(j)} = b^{(j)} - \alpha \frac{\partial}{\partial b^{(j)}} J(w, b, x)$$

$$x_k^{(i)} = x_k^{(i)} - \alpha \frac{\partial}{\partial x_k^{(i)}} J(w, b, x)$$

代码片段

在 *TensorFlow* 中，有一个非人技术——“*Auto Diff*”，仅给出 $J(w,b)$ ，剩下的让 *Tensorflow* 来助你完成吧！

假如 $J = (wx - 1)^2, f(x) = wx$ ，则自微分代码如下：

```
w = tf.Variable(3.0) # Variable() means the parament that we want to optimize
x = 1.0
y = 1.0
alpha = 0.01

iterations = 30
for iter in range(iterations):
    with tf.GradientTape() as tape:
        f = w * x
        costJ = (f - y)**2

    [dJdw] = tape.gradient(costJ, [w])

    w.assign_add(-alpha * dJdw)
```

以下是在 *TensorFlow* 中的协同过滤实现：

```
# Adam Optimizer
optimizer = keras.optimizers.Adam(learning_rate=1e-3)

iterations = 200
for iter in range(iterations):
    with tf.GradientTape() as tape:
        cost_value = cofiCostFuncV(X, W, b, Ynorm, R, num_uers, num_movies, lambda)

    grads = tape.gradient(cost_value, [X, W, b])

    optimizer.apply_gradients(zip(grads, [X, W, b]))
```

二元标签

有时可能不是给电影评分，而仅是标记是“神”还是“烂”片，此时从回归问题转化成归类问题，不再啰嗦。

Movie	Alice(1)	Bob(2)	Carol(3)	Dave(4)
Love at last	1	1	0	0
Romance forever	1	?	?	0
Cute puppies of love	?	1	0	?
Nonstop car chases	0	0	1	1
Swords vs. karate	0	0	1	?

$$y^{(i,j)} : f_{w,b,x}(x) = g \left((w^{(j)}) \cdot x^{(i)} + b^{(j)} \right)$$

$$L(f_{w,b,x}(x), y^{(i,j)}) = -y^{(i,j)} \log (f_{w,b,x}(x)) - \left(1 - y^{(i,j)} \right) \log (1 - f_{w,b,x}(x))$$

$$J(w,b,x) = \sum_{(i,j):r(i,j)=1} L(f_{w,b,x}(x), y^{(i,j)})$$

其中 $g(z) = \frac{1}{1+e^{-z}}$ 。

均值归一化 (Mean Normalization)

假如有个新人 *Eve* 来了，由于他没有对任何电影做出任何评价，所以原先的模型训出的 $w^{(j)}$ 和 $b^{(j)}$ 将等于 0，所以 $\hat{y}^{(i,5)} = \vec{w}^{(5)} \cdot \vec{x}^{(i)} + b^{(5)} = 0$ ，显然不合实际，所以得进行均值归一化。

Movie	Alice(1)	Bob(2)	Carol(3)	Dave(4)	Eve(5)
Love at last	1	1	0	0	?
Romance forever	1	?	?	0	?
Cute puppies of love	?	1	0	?	?
Nonstop car chases	0	0	1	1	?
Swords vs. karate	0	0	1	?	?

首先，将评价值写成矩阵形式：

$$\begin{bmatrix} 5 & 5 & 0 & 0 & ? \\ 5 & ? & ? & 0 & ? \\ ? & 4 & 0 & ? & ? \\ 0 & 0 & 5 & 4 & ? \\ 0 & 0 & 5 & 0 & ? \end{bmatrix}$$

求每个电影的平均评价值：

$$\mu = \begin{bmatrix} 2.5 \\ 2.5 \\ 2 \\ 2.25 \\ 1.25 \end{bmatrix}$$

所有电影的评价值-对应的平均评价值，得到归一化矩阵：

$$\begin{bmatrix} 2.5 & 2.5 & -2.5 & -2.5 & ? \\ 2.5 & ? & ? & -2.5 & ? \\ ? & 2 & -2 & ? & ? \\ -2.25 & -2.25 & 2.75 & 1.75 & ? \\ -1.25 & -1.25 & 3.75 & -1.25 & ? \end{bmatrix}$$

归一化矩阵训出来的 $w^{(j)}$ 和 $b^{(j)}$ 也等于 0，但此时 $\hat{y}^{(i,5)} = \vec{w}^{(5)} \cdot \vec{x}^{(i)} + b^{(5)} + \mu^{(i)} = 2.5$ ，所以归一化后的推荐值写作：

$$\hat{y}^{(i,j)} = \vec{w}^{(j)} \cdot \vec{x}^{(i)} + b^{(j)} + \mu^{(i)}$$

内容过滤法

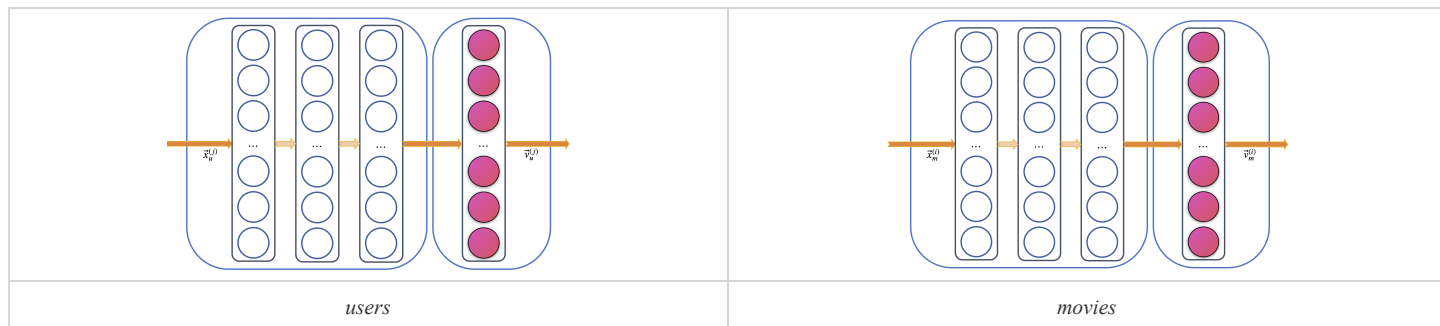
如果某人在过去评高分某个电影，则向他推荐与它**特征相似**的其他电影，即给每个人搞一个“**兴趣画像**”，由他过去评高分的电影的特征来形成的。

假如人（年龄、地区、性别、评分等）和电影（年份、影评、类别等）都有自己的特征，即 $x_u^{(j)}$ 和 $x_m^{(i)}$ ，由这两个特征（向量）可以进一步提取人和电影的匹配（偏好）向量 $v_u^{(j)}$ 和 $v_m^{(i)}$ ，则：

$$\hat{y}^{(i,j)} = \vec{v}_u^{(j)} \cdot \vec{v}_m^{(i)}$$

偏好提取

从 $x_u^{(j)}$ 到 $v_u^{(j)}$ ，则借助老朋友“*neural network*”来完成，保证 $v_u^{(j)}$ 和 $v_m^{(i)}$ 大小一样即可。



之后，将两张网组合，合并共用一个 $J(W, B)$ ：

$$J(W, B) = \sum_{(i,j):r(i,j)=1} \left(v_u^{(j)} \cdot v_m^{(i)} - y^{(i,j)} \right)^2 + \text{NN Regularization Term}$$

代码片段

以下是在 *TensorFlow* 中的内容过滤实现：

```

user_NN = tf.keras.model.Sequential([
    tf.keras.layers.Dense(256, activations='relu'),
    tf.keras.layers.Dense(128, activations='relu'),
    tf.keras.layers.Dense(32)
])
item_NN = tf.keras.model.Sequential([
    tf.keras.layers.Dense(256, activations='relu'),
    tf.keras.layers.Dense(128, activations='relu'),
    tf.keras.layers.Dense(32)
])

xu = tf.keras.layers.Input(shape=(num_user_features))
vu = user_NN(xu)
vu = tf.linalg.l2_normalize(vu, axis=1) # length normalization

xm = tf.keras.layers.Input(shape=(num_item_features))
vm = item_NN(xm)
vm = tf.linalg.l2_normalize(vm, axis=1)

y = tf.keras.layers.Dot(axes=1)([vu, vm])

model = Model([vu, vm], y)

cost_fn = tf.keras

```

强化学习

Reinforcement Learning 是一种范式，其中一个智能体 (**Agent**) 通过执行动作 (**Actions**) 来与环境 (**Environment**) 进行互动。每次动作后，得到一个新的状态 (**State**) 和一个奖励 (**Reward**)，目标就是得到一个最优的策略 (**Policy**)，得在长期互动中（未来）获得的最大化回报。

核心过程

核心过程: **state** $\longrightarrow$ **action** $\longrightarrow$ **new state** $\longrightarrow$ **reward**, 即 $(s, a, R(s), s')$

折扣因子 (Discount Factor)

在“耗时”与价值之间权衡，引入折扣因子 ($0 \leq \gamma \leq 1$) 来降低长耗时带来的回报， γ 越小，智能体越关注即时回报，反之越关注长远回报。

从时间 t 后 k 步的总回报: $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$

策略 (Policy, π)

Policy 由当前状态得到下一步的动作，可以恒定的 ($a = \pi(s)$) 亦可以随机的 ($P(a|s) = \pi(s, a)$)。

回报

Action Value Function, $Q(s, a)$ ，即在状态 s 下，采取动作 a 后，下一个状态 s' 共可获得的最优回报，所以一旦有了 $Q(s, a)$ 的所有值，则下一步 a 选 $Q(s, a)$ 最大的那个动作 a ，即可获得长远最大回报。

State Value Function, $V(s)$ ，从状态 s 起所能获得的长远最大回报。

Bellman 方程

$$Q(s, a) = R(s) + \gamma \max_{a'} Q(s', a')$$

直观上，可以将 G_t 与 *Bellman Equation* 关联得到：

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \\ &= R_{t+1} + \gamma (R_{t+2} + \gamma R_{t+3} + \dots) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned}$$

Sometimes, due to the randomness of the environment, an agent's actions may not always achieve the exact expected outcome.

In such cases, the expected optimal return, i.e., $E(Q(s,a))$, is often used instead of the optimal return from a single execution.

For this reason, the Bellman equation is slightly adjusted :

$$Q(s,a) = R(s) + \gamma E[\max_{a'} Q(s',a')]$$

阿波罗计划

月球登陆计划，现在启动！

状态 $s = \begin{bmatrix} x \\ y \\ \dot{x} \\ \dot{y} \\ \theta \\ \dot{\theta} \\ l \\ r \\ a \end{bmatrix}$, 其中 $x,y,\dot{x},\dot{y},\theta,\dot{\theta} \in \mathbb{R}$, 而 $l,r \in \{0,1\}$ 左/右脚着地; $\gamma = 0.985$ 。

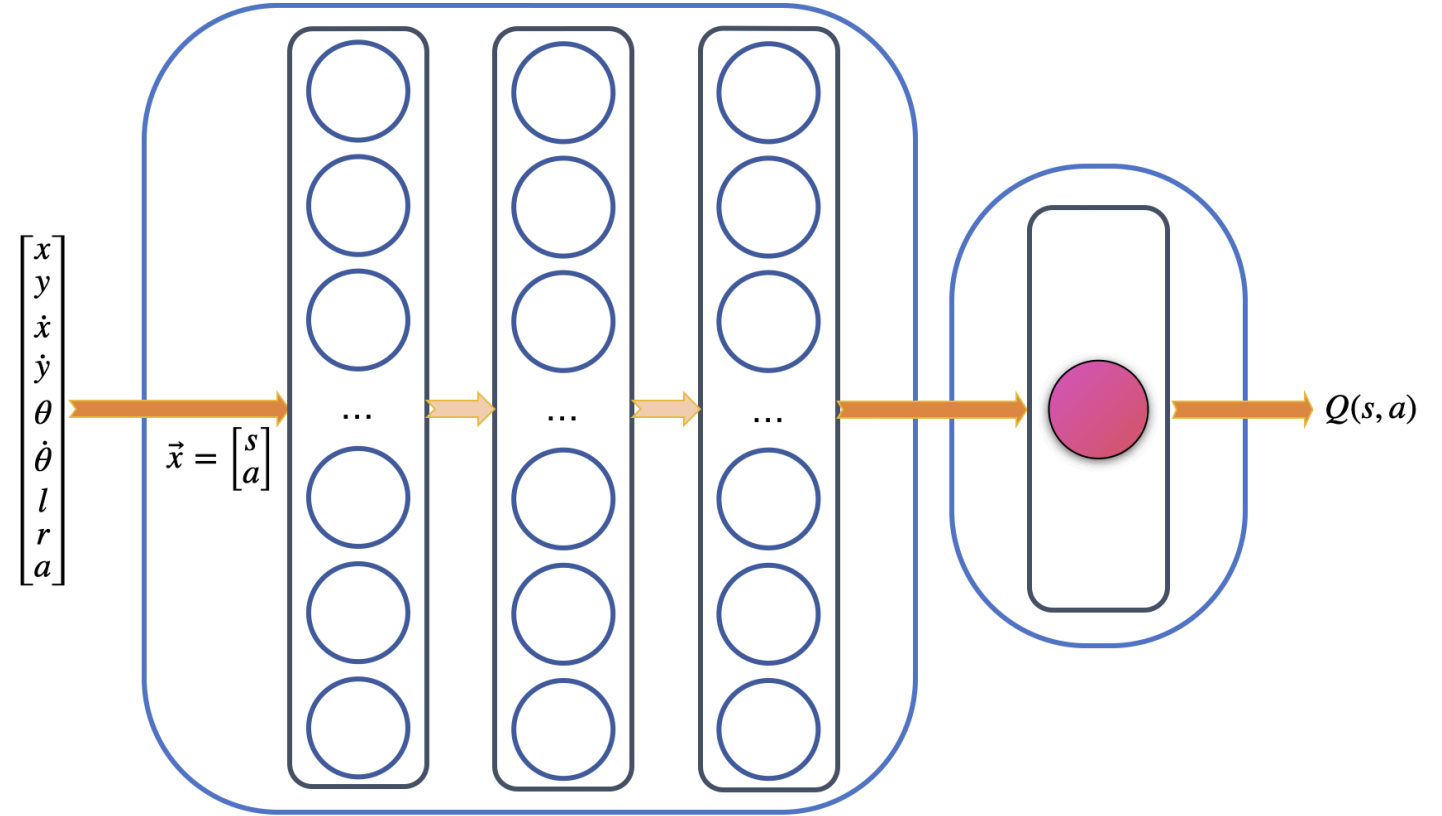
Event	Reward
Getting to Landing Pad	100 to 140
Crash	-100
Soft Landing	+100
Leg Grounded	+10
Fire Main Engine	-0.3
Fire Side Thruster	-0.03

动作 a 采取四位独热编码，即无（1000）、左喷气（0100）、主喷气（0010）、右喷气（0001）。

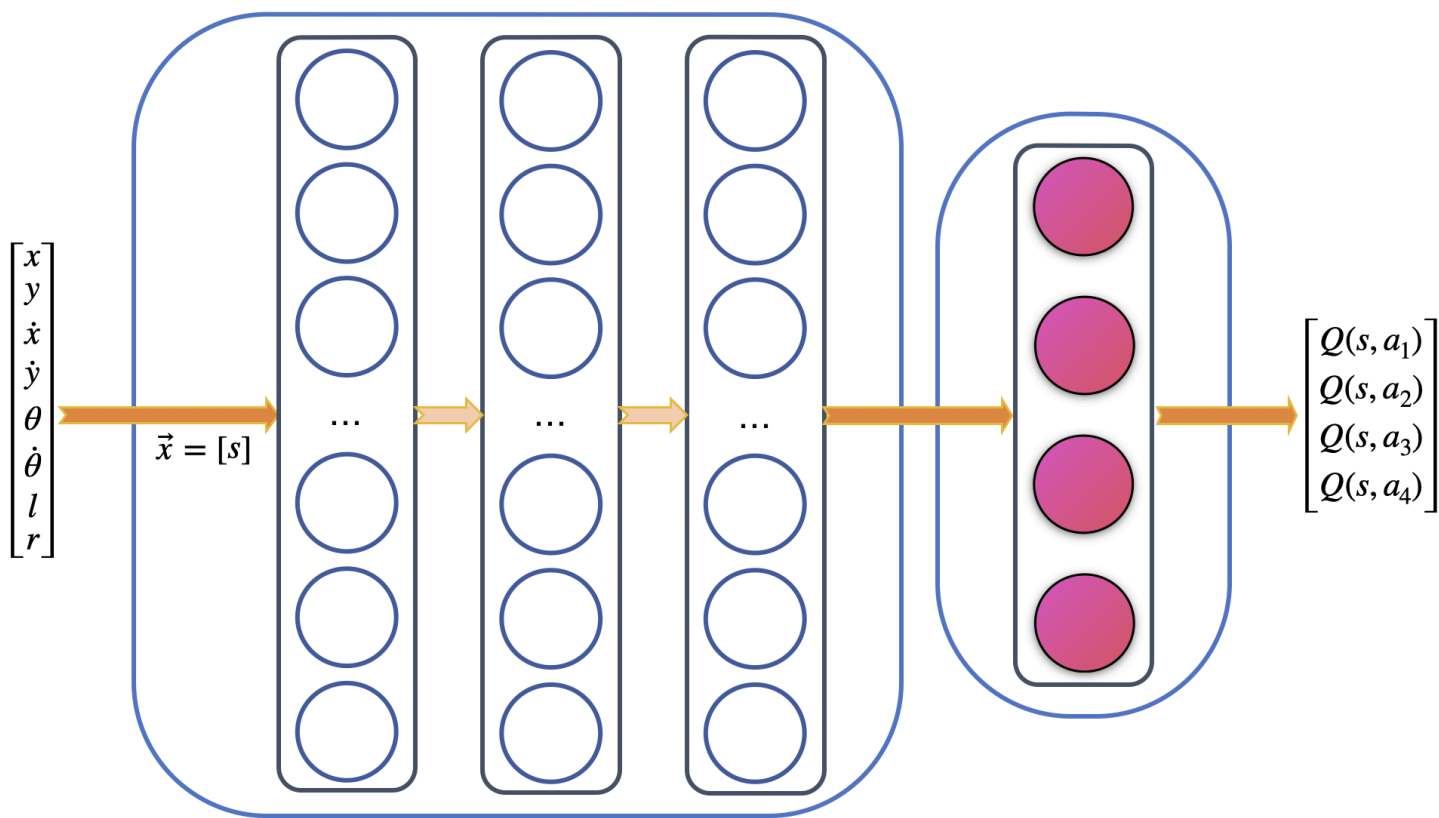
到此，又得让老朋友 *neural network* 来助力，先随机大量行为，得到众多四元组 $(s^{(i)}, a^{(i)}, R(s^{(i)}), s'^{(i)})$, $i = 1, 2, 3, \dots$ 。

$$\begin{cases} y^{(1)} &= R(s^{(1)}) + \gamma \max_{a'} Q(s'^{(1)}, a') \\ y^{(2)} &= R(s^{(2)}) + \gamma \max_{a'} Q(s'^{(2)}, a') \\ y^{(3)} &= R(s^{(3)}) + \gamma \max_{a'} Q(s'^{(3)}, a') \\ &\vdots \end{cases}$$

训练的细节不再研究，反正由 $(s^{(i)}, a^{(i)}, R(s^{(i)}), s'^{(i)})$ 可以得到 $(x^{(i)}, y^{(i)})$ 来训练 $Q(a, s)$ ，起初 $Q(s, a)$ 是完全随机出来的，后来逐步可以近似 $Q(s, a) \approx y$ 。



以上 *neural network* 每次只能训练一个行为，低能，稍微修正一下输入/出：



1. 标准术语集

Probability Density: 概率密度

Reinforcement Learning: 强化学习

Value Function: 价值函数

State Value Function: 状态价值函数

Action Value Function: 动作价值函数

